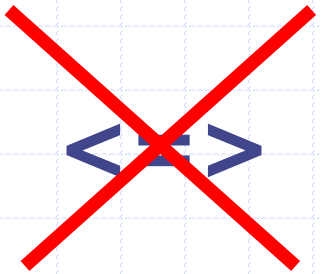
Пројектовање синтаксног анализатора 1

Граматика **=>** **Језик**

(Бесконачан) скуп реченица

Граматика  **Језик**

(Бесконачан) скуп реченица

Различите граматике могу дефинисати исти језик

S -> E
E -> T
T -> iks
E -> E plus T

S -> E
E -> T
T -> iks
E -> T plus E

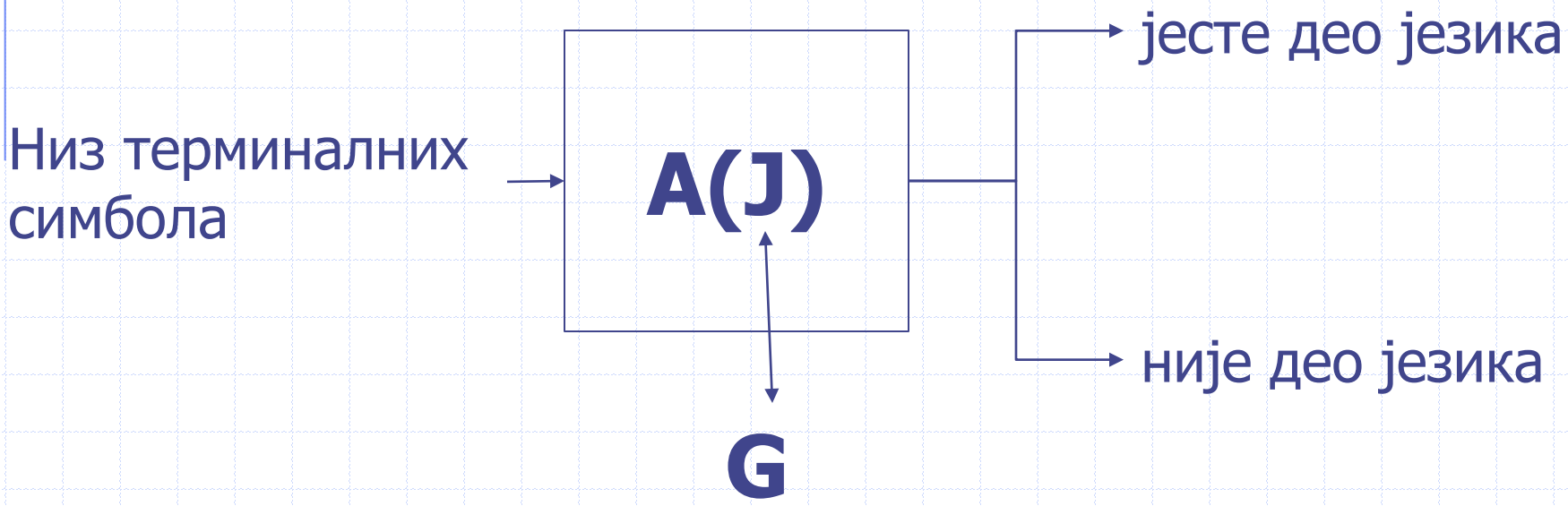
S -> E
E -> T
T -> iks
E -> E plus E

S -> E
E -> iks
E -> E plus E

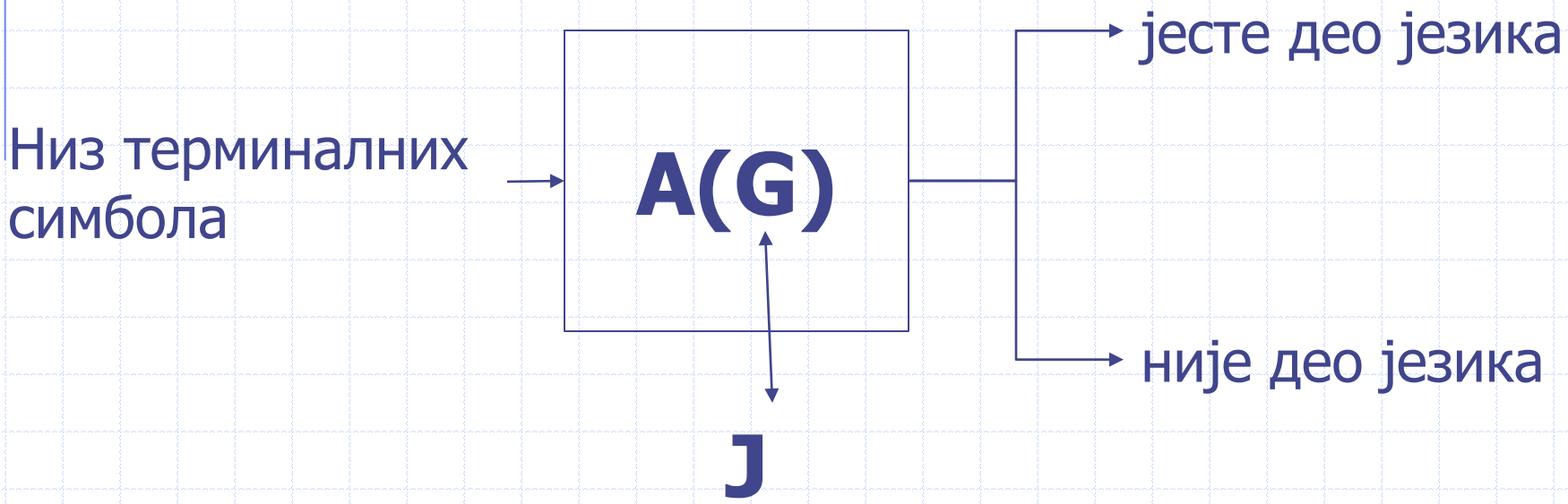
S -> iks
S -> S plus iks

Језик:
iks
iks plus iks
iks plus iks plus iks
iks plus iks plus iks plus iks
...

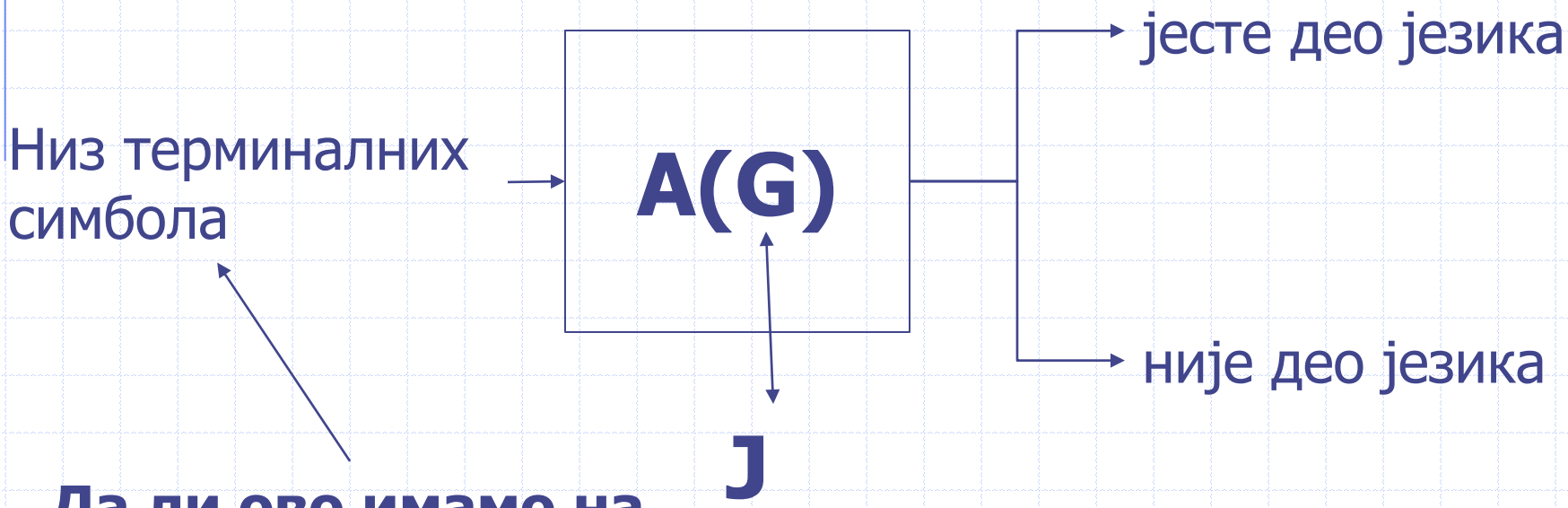
Синтаксни анализатор



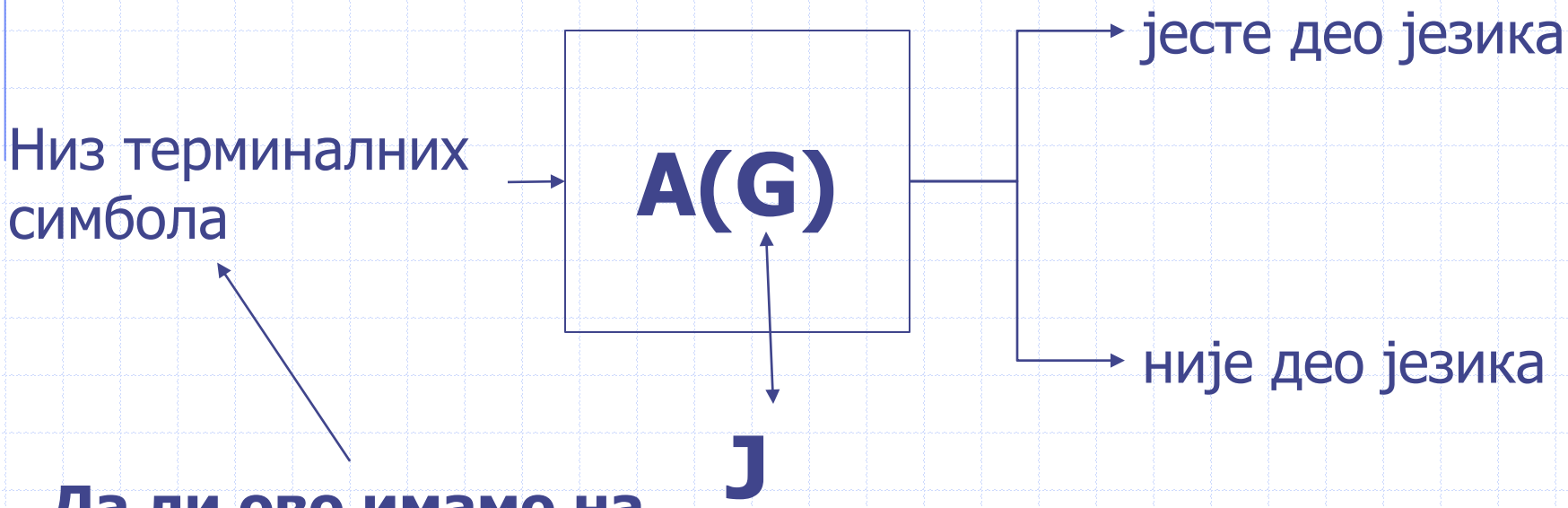
Синтаксни анализатор



Синтаксни анализатор



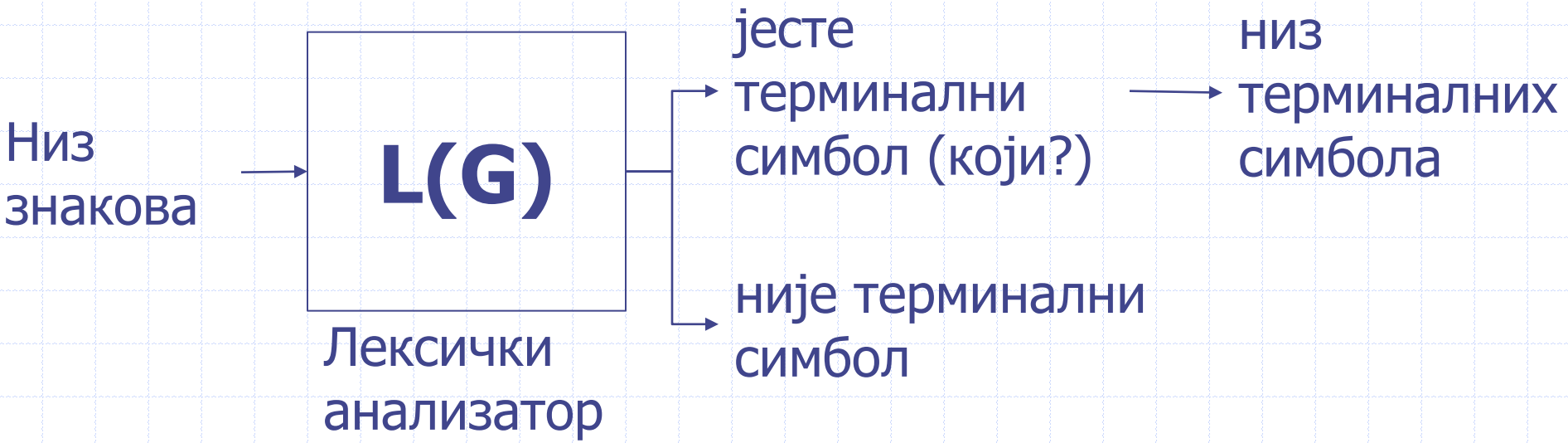
Синтаксни анализатор



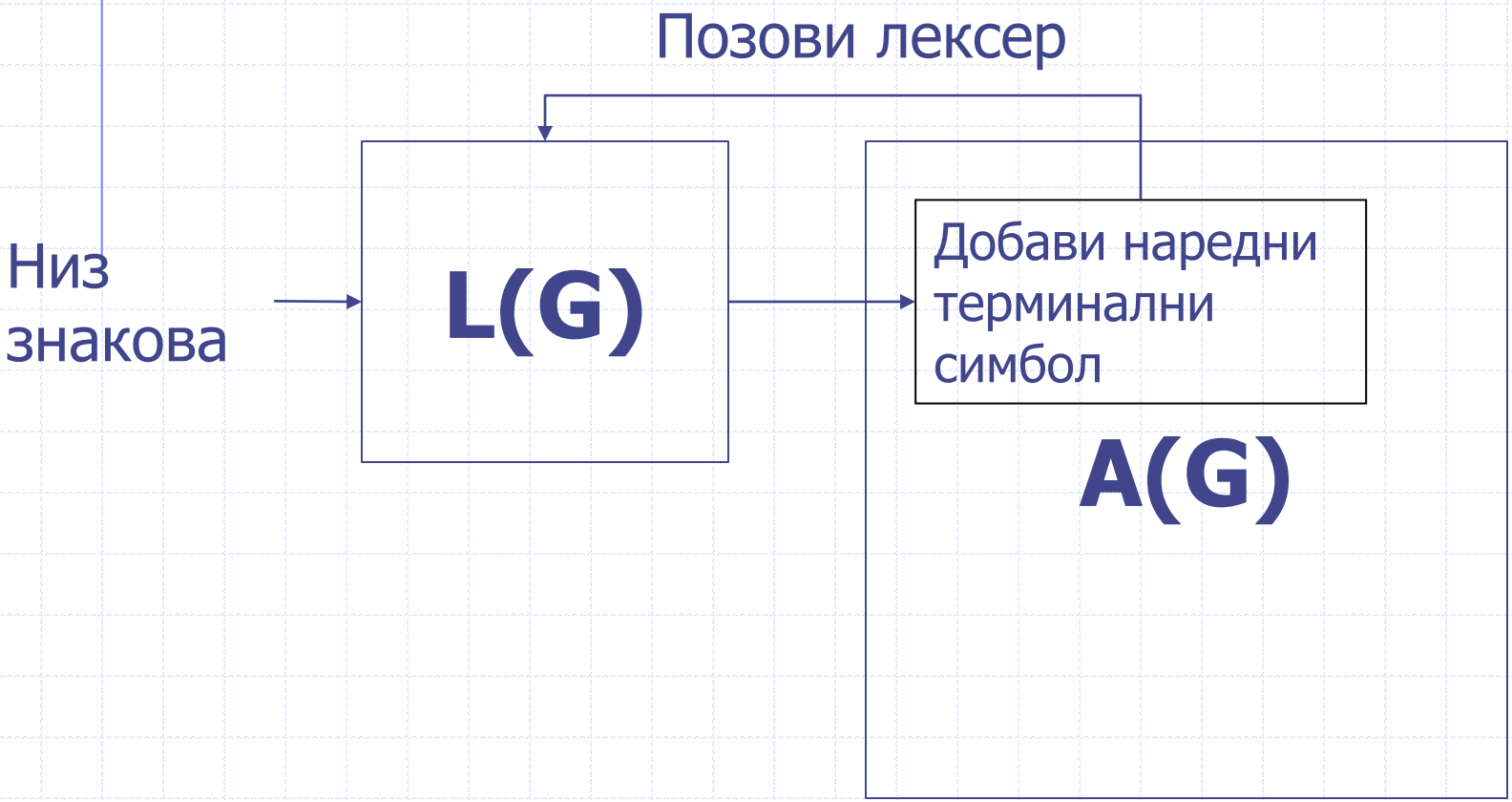
Да ли ово имамо на улазу? – Не, имамо низ знакова

Синтаксни анализатор

- Два приступа:
  - Написати граматику језика код које су знакови терминални симболи.
  - Анализу обавити у два корака:



Лексички анализатор



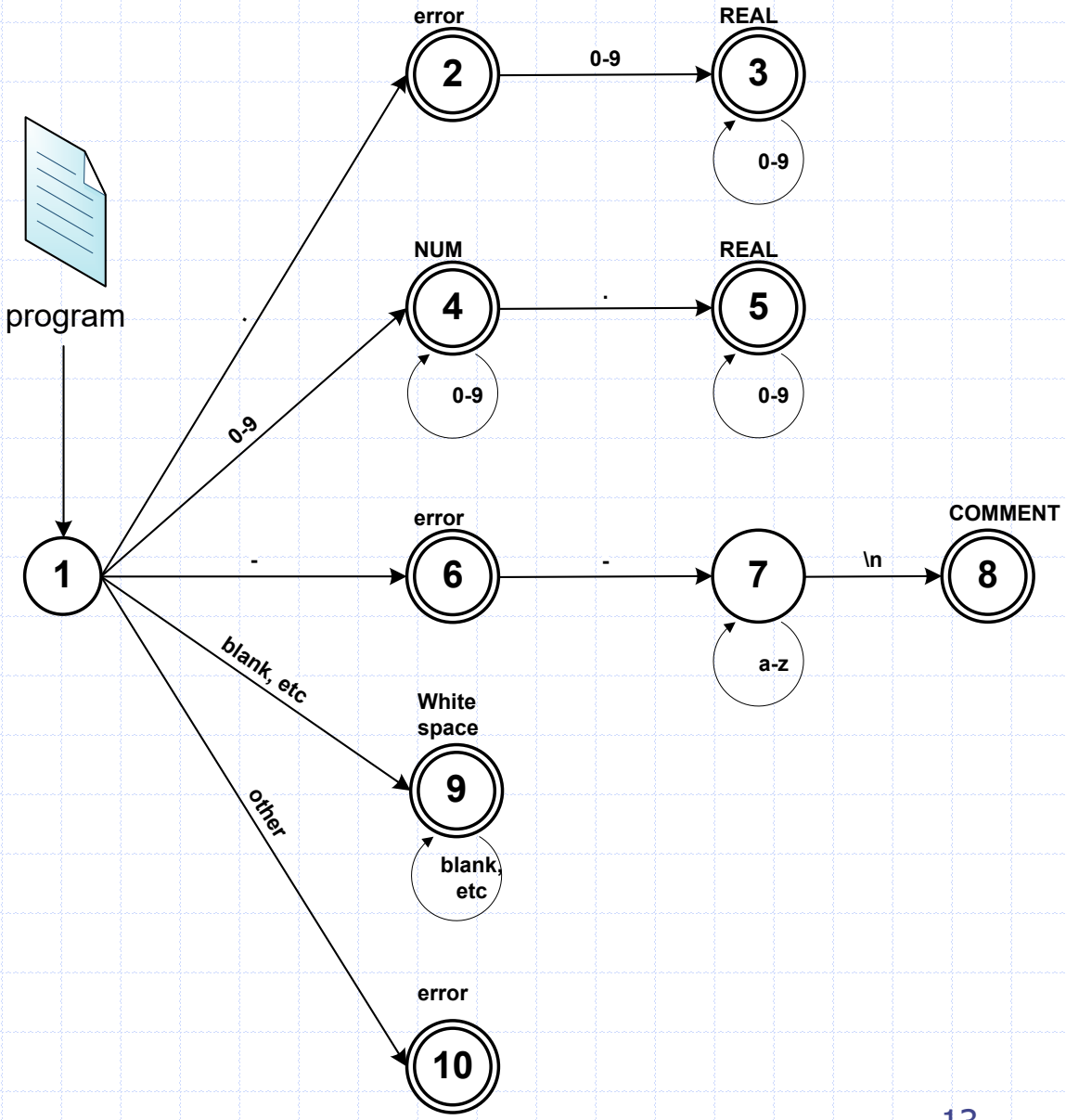
Лексички анализатор

- Слично као синтаксни анализатор
- Другачија граматика - једноставнија
  - Терминални симболи су знаци
- Регуларна граматика (нижег нивоа од контекстно слободних граматика):
  G (N, T, Σ, P)
  - Где је P скуп смена (продукција) које могу имати један од следећа три облика:
    B -> α, где B припада N, а α припада T
    B -> αC (или B -> Cα), где C припада N
    B -> ε, где је ε празан знаковни низ

Лексички анализатор

- Алгоритам за одређивање да ли је неки улаз део језика дефинисаног регуларном граматиком (лексички исправан) заснива се на детерминистичком аутомату са коначним бројем стања (DFA – Deterministic Finite Automata).
- То је јако повољан и ефикасан алгоритам.

Лексички анализатор



Прављење лексичког анализатора

- Можемо га правити ручно
  - Коришћењем DFA
  - Или неком другом, мање ефикасном техником (али опет довољно ефикасном)
- Можемо га генерисати помоћу алата
  - Алат који на основу спецификације генерише лексички анализатора
  - Један од таквих алата је **Flex**
  - **Flex** генерише анализатор који се заснива на DFA

Прављење лексичког анализатора

- Како специфицирамо терминале језика?
  - тј. регуларну граматику којом су дефинисани?
- Не пишемо продукције
  - Много напорно и нејасно
- Користимо регуларне изразе
  - Чест механизам за претраживање текста

Нотација регуларних израза

a	Обичан знак који стоји сам за себе
ε	Празан стринг
	Такође празан стринг
M | N	Избор M или N
M · N	Спајање M и N, иза M иде N
MN	Други начин да се напише спајање M и N
M*	Понављање M, нула или више пута
M+	Понављање M, један или више пута
M?	Једно или ни једно појављивање M
[a-zA-Z]	Избор знака из скупа знакова
.	Тачка, означава један знак, било који, осим новог реда
“abc”	Низ знакова, било којих знакова

Коришћење регуларних израза

- Коришћењем овог језика можемо специфицирати лексичке симболе (токене) програмских језика
  - Кључна реч „if“
  - Идентификатор (почиње словом, а затим могу и слова и цифре)
  - Цели бројеви
  - Позитивни реални бројеви (53.7 .28 4.)

Коришћење регуларних израза

- Коришћењем овог језика можемо специфицирати лексичке симболе (токене) програмских језика
  - Кључна реч „if“

    if
  - Идентификатор (почиње словом, а затим могу и слова и цифре)

    [a-z][a-z0-9]*
  - Цели бројеви

    “-”?[0-9]+
  - Позитивни реални бројеви (53.7 .28 4.)

    ([0-9]+”.”[0-9]*) | ([0-9]*”.” [0-9]+)

Flex: генератор лексичких анализатора

Генерише Це код на основу лексичке спецификације

```
%{
   int brojDjura = 0;
%}

%%
djura brojDjura += 1;
.      ;
```

```
#include <stdio.h>
int main() {
   yylex();
   printf("%d", brojDjura);
   return 0;
}
```

Flex: генератор лексичких анализатора

Генерише Це код на основу лексичке спецификације

```
%{
%}

%%
djura return 1;
pera  return 2;
mika  return 3;
```

```
#include <stdio.h>
int main() {
  int tokType = yylex();
  while (tokType != 0) {
    print("%d", tokType);
    tokType = yylex();
  }
  return 0;
}
```

Flex: генератор лексичких анализатора

Генерише Це код на основу лексичке спецификације

```
%{
%}

%%
djura return 1;
pera  return 2;
mika  return 3;
```

```
#include <stdio.h>
int main() {
  int tokType = yylex();
  while (tokType != 0) {
    print("%s", yytext);
    tokType = yylex();
  }
  return 0;
}
```

Flex: генератор лексичких анализатора

Генерише Це код на основу лексичке спецификације

```
%{
%}

%%
djura return 1;
pera  return 2;
mika  return 3;
```

```
#include <stdio.h>
int main() {
  yyin = fopen("datoteka.txt", "r");
  int tokType = yylex();
  while (tokType != 0) {
    print("%s", yytext);
    tokType = yylex();
  }
  return 0;
}
```

Flex: генератор лексичких анализатора

Генерише Це код на основу лексичке спецификације

```
%{
%}

%%
djura return 1;
pera  return 2;
mika  return 3;
```

```
#include <stdio.h>
int main() {
  yyin = fopen("datoteka.txt", "r");
  int tokType = yylex();
  while (tokType != 0) {
    print("%s, %d", yytext, yylen);
    tokType = yylex();
  }
  return 0;
}
```

Flex: генератор лексичких анализатора

Генерише Це код на основу лексичке спецификације

- За сваки тип лексичког симбола наводи се
  - Регуларни израз и акција у Це језику, нпр.

```
%{ /* C declarations */
#include "tokens.h" /* definitions of IF, ID, NUM, … */

struct {int ival; string sval; double fval;} yylval;


%}

digits [0-9]+ /* Lex definition of digits */

%% /* Regular expressions and actions */
if    {       return IF;}
[a-z][a-z0-9]*    {       yylval.sval = String(yytext); return ID;}
digits    {       yylval.ival = atoi(yytext); return NUM;}
(digits"."[0-9]*) | ([0-9]*"."digits)
    {       yyval.fval = atof(yytext); return FLOAT;}
("--"[a-z]*"\n") | (" " | "\n" | "\t")+   {       /* do nothing */ }
.    {       error();}
```

Flex: генератор лексичких анализатора

Генерише Це код на основу лексичке спецификације

- За сваки тип лексичког симбола наводи се
  - Регуларни израз и акција у Це језику, нпр.

```
%{ /* C declarations */
#include "tokens.h" /* definitions of IF, ID, NUM, … */
#include "errormsg.h"
struct {int ival; string sval; double fval;} yylval;
int charPos = 1;
void adj() {EM_tokPos = charPos; charPos += yyleng;}
%}

digits [0-9]+ /* Lex definition of digits */

%% /* Regular expressions and actions */
if    {adj(); return IF;}
[a-z][a-z0-9]*    {adj(); yylval.sval = String(yytext); return ID;}
digits    {adj(); yylval.ival = atoi(yytext); return NUM;}
(digits"."[0-9]*) | ([0-9]*"."digits)
    {adj(); yyval.fval = atof(yytext); return FLOAT;}
("--"[a-z]*"\n") | (" " | "\n" | "\t")+   {adj(); /* do nothing */ }
.    {adj(); error();}
```

Детаљи Flex спецификације и излаза

- Излаз Flex-а је Це програм lex.yy.c
  - Који интерпретира DFA и извршава Це код акција
- Спецификација
  - I део, између %{ и %}: Це #include и декларације
    - Потребне за Це код до краја датотеке
  - II део: скраћенице рег. израза и декларације стања
    - Нпр. **digits** је скраћеница за **[0-9]+**
  - III део: регуларни изрази и акције
    - Акције враћају целобројну вредност (int), која означава врсту пронађеног симбола
- Даље читање: Flex упутство на Интернету

Пример вишег програмског језика
Tiger – аутор A.W.Apple, Modern Compiler Impl.

- Tiger је мали језик са
  - Угњежденим функцијама
  - Вредностима слогова са имплицитним показивачима
  - Низовима
  - Целобројним и стринг променљивама
  - Неколико конструкција за структуирану контролу тока
- Спецификација:
  - Лексика: уобичајени идентификатори и Це коментари
  - Декларације и променљиве и изрази

Пример: програм queens (1/2)

```
/* A program to solve the 8-queens problem */
let
  var N := 8

  type intArray = array of int

  var row := intArray [ N ] of 0
  var col := intArray [ N ] of 0
  var diag1 := intArray [N+N-1] of 0
  var diag2 := intArray [N+N-1] of 0

  function printboard() =
    (for i := 0 to N-1 do
       (for j := 0 to N-1 do
          print(if col[i]=j then " O" else " .");
        print("\n"));
     print("\n"))
```

Пример: програм queens (2/2)

```
function try(c:int) =
  if c = N
  then printboard()
  else for r := 0 to N-1 do
        if row[r] = 0 & diag1[r+c] = 0 & diag2[r+7-c] = 0
          then (row[r] := 1;
                diag1[r+c] := 1;
                diag2[r+7-c] := 1;
                col[c] := r;
                try(c+1);
                row[r] := 0;
                diag1[r+c] := 0; diag2[r+7-c] := 0)

in try(0)
end
```

Декларације (1/3)

- *decs* је низ декларација типова, вредности и функција
  - *decs* $\rightarrow$ {*dec*}, где {*x*} може бити празан скуп ε
  - *dec* $\rightarrow$ *tydec*
    - $\rightarrow$ *vardec*
    - $\rightarrow$ *fundec*
- Типови података
  - *tydec* $\rightarrow$ **type** *type-id* = *ty*
  - *ty* $\rightarrow$ *type-id*
    - $\rightarrow$ {*tyfields*} овде су { и } терминални симболи
    - $\rightarrow$ **array of** *type-id*
  - *tyfields* $\rightarrow$ ε
    - $\rightarrow$ *id*: *type-id*{, *id* : *type-id*}

Декларације (2/3)

- Уграђени типови: int и string
- Свака декларација низа или слога ствара нов тип
  - Чак иако су сва поља идентична са неким претходним
- Међусобно рекурзивни типови
  - Сваки циклус рекурзије мора проћи кроз тип слога или низа
  - Нпр. саморекурзија: type intlist = {head:int, tail:intlist}
  - Међурекурзија:

    type tree = {key:int, children:treelist}

    type treelist = {head:tree, tail:treelist}
- Декларације променљивих:

  vardec → **var** *id* := *exp*

  → **var** *id*:*type-id* := *exp*

Декларације (3/3)

- Декларације функција:
  - *fundec* → **function** *id* (*tyfields*) = *exp*
  - *fundec* → **function** *id* (*tyfields*) : *type-id* = *exp*
- Међусобно рекурзивне процедуре и функције, нпр.

```
function treeLeaves(t : tree) : int =
    if t=nil then 1
    else treelistLeaves(t.children)
function treelistLeaves(L : treelist) : int =
    if L=nil then 0
    else treeLeaves(L.head) + treelistLeaves(L.tail)
```

Променљиве и изрази (1/4)

- Л-вредност је локација која се чита или у коју се пише

lvalue → id		променљиве
	→ lvalue . id	поља слогова (структура)
	→ lvalue [exp]	елементи низова

- Изрази
  - lvalue
  - Изрази без вредности
    - Позив процедуре, додела, if-then, while, break, и понекад if-then-else
  - nil – нула, припада сваком типу слога
  - Низ израза (*exp*; *exp*; ... *exp*) – израчунава изразе с лева у десно – коначна вредност је вредност последњег израза у низу
  - Нема вредност: () и **let** израз без ичега између **in** и **end**

Променљиве и изрази (2/4)

- Целобројни литерал – низ децималних цифара
- Стринг литерал – низ знакова између наводника “”
- Специјални знаци се наводе након знака “\” (\n, \t, итд.)
- Негација – целобројном изразу може претходити знак “-”
- Позив функције: *id*() или *id*(*exp*{,*exp*})
- Аритметички израз: *exp op exp*, gde *op* +, -, *, /, захтевају и производе целе бројеве
- Поређење: *exp op exp*, gde op =, <>, >, <, >=, <=, пореде операнде и производе 1 (тачно) или 0 (нетачно)
- Поређење стрингова: два стринга су иста ако им је садржај идентичан

Променљиве и изрази (3/4)

- Изрази са Буловим операторима: *exp op exp*, где је *op* & или |
- Приоритет оператора: unarni -, *, /, +, -, =, <>, >, <, >=, <=, &, |
- Асоцијативност оператера: *, /, +, -, су асоцијативни у лево; оператори поређења нису асоцијативни.
- Додела вредности: *lvalue* **:=** *exp*
- If-then-else: **if** *exp1* **then** *exp2* **else** *epx3*, где *exp2* и *exp3* морају бити истог типа, тај тип је и тип резултата
- If-then: **if** *exp1* **then** *exp2*, где *exp2* не производи вредност
- While: **while** *exp1* **do** *exp2*, где *exp2* не производи вредност
- For: **for** *id* := *exp1* **to** *exp2* **do** *exp3*, *exp3* нема вредност

Променљиве и изрази (4/4)

- Break: прекида извршење најближе петље (while или for)
- Let: израз **let** *decs* **in** *expseq* **end** евалуира декларације *decs*, а затим израчунава низ израза *expseq*, чији коначни резултат је резултат задњег израза у низу.
- Заграде: могу се користити за синтаксно груписање израза

Лексички анализатор за Tiger (1/4)

tokens.h

```
typedef struct  {
            int pos;
            int ival;
            string sval;
            } YYSTYPE;
extern YYSTYPE yylval;

# define ID 257
# define STRING 258
# define INT 259
# define COMMA 260
# define COLON 261
# define SEMICOLON 262
# define LPAREN 263
# define RPAREN 264
# define LBRACK 265
# define RBRACK 266
# define LBRACE 267
# define RBRACE 268
# define DOT 269
```

```
# define PLUS 270
# define MINUS 271
# define TIMES 272
...
# define IF 284
# define THEN 285
# define ELSE 286
# define WHILE 287
# define FOR 288
# define TO 289
# define DO 290
# define LET 291
# define IN 292
# define END 293
# define OF 294
# define BREAK 295
# define NIL 296
# define FUNCTION 297
# define VAR 298
# define TYPE 299
```

Лексички анализатор за Tiger (2/4)

tiger.lex (1/2)

```
...
%%
" "         {adjust(); continue;}
\n          {adjust(); EM_newline(); continue;}
\t          {adjust(); EM_newline(); continue;}
","         {adjust(); return COMMA;}
":"         {adjust(); return COLON;}
";"         {adjust(); return SEMICOLON;}
"("         {adjust(); return LPAREN;}
")"         {adjust(); return RPAREN;}
"["         {adjust(); return LBRACK;}
"]"         {adjust(); return RBRACK;}
"{"         {adjust(); return LBRACE;}
"}"         {adjust(); return RBRACE;}
"."         {adjust(); return DOT;}
"+"         {adjust(); return PLUS;}
"-"         {adjust(); return MINUS;}
"*"         {adjust(); return TIMES;}
"/"         {adjust(); return DIVIDE;}
"="         {adjust(); return EQ;}
...
```

Лексички анализатор за Tiger (3/4)

tiger.lex (2/2)

```
...
while       {adjust(); return WHILE;}
for         {adjust(); return FOR;}
to          {adjust(); return TO;}
break       {adjust(); return BREAK;}
let         {adjust(); return LET;}
in          {adjust(); return IN;}
end         {adjust(); return END;}
function    {adjust(); return FUNCTION;}
var         {adjust(); return VAR;}
type        {adjust(); return TYPE;}
array       {adjust(); return ARRAY;}
if          {adjust(); return IF;}
then        {adjust(); return THEN;}
else        {adjust(); return ELSE;}
do          {adjust(); return DO;}
of          {adjust(); return OF;}
nil         {adjust(); return NIL;}
[a-zA-Z][a-zA-Z0-9"_"]*   {adjust(); yylval.sval=String((char*)yytext);
                           return ID;}
...
```

Лексички анализатор за Tiger (3/4)

driver.c

```
int main(int argc, char** argv) {
  char* fname = argv[1];
  EM_reset(fname);
  yyin = fopen(fname, "r");
  while (1) {
    int tok = yylex();
    if (tok == 0) break;
    switch(tok) {
    case ID:
    case STRING:
      printf("%10s %4d %s\n", tokname(tok), EM_tokPos, yylval.sval);
      break;
    case INT:
      printf("%10s %4d %d\n", tokname(tok), EM_tokPos, yylval.ival);
      break;
    default:
      printf("%10s %4d\n", tokname(tok), EM_tokPos);
    }
  }
  fclose(yyin);
  return 0;
}
```

Лексички анализатор за Tiger (4/4)

Излаз: прог. queens трансформисан у низ симбола

```
FUNCTION  210
ID  219 printboard
LPAREN  229
RPAREN  230
EQ  232
LPAREN  241
FOR  242
ID  246 i
ASSIGN  248
INT  251 0
TO  253
ID  256 N
MINUS  257
INT  258 1
DO  262
LPAREN  265
FOR  266
ID  270 j
...
```

```
function printboard() =
    (for i := 0 to N-1
        do (for j := 0 to N-1
            do print(if col[i]=j then " O" else " .");
           print("\n"));
     print("\n"))
...
```